

Gestione Utenti

(sola visualizzazione)

Riepilogo della consegna

Esame “Progettazione Full Stack”

Prof. Bianchini — UniBS

Appello **20260629**

*Documento di studio: raccoglie **tutte** le modifiche fatte rispetto al progetto originale per soddisfare i tre esercizi della consegna.*

30 giugno 2026

Indice

1	La consegna in breve	2
2	Esercizio 1 — Modello WebML (documentazione)	2
2.1	Premesse e requisiti	2
2.2	Progetto coarse — Area Utenti	3
2.3	Progetto dettagliato — suddivisione in pagine	3
2.4	Specifica di dettaglio della pagina «Lista Utenti»	3
2.4.1	Diagramma WebML	3
2.4.2	View component — <code>UtentiUnit (MultipleDetails)</code>	4
2.4.3	Operazione — <code>Delete(User)</code> (opzionale)	4
2.4.4	Link uscenti dalla pagina	4
2.5	Layout di pagina (navbar + utente corrente)	5
2.6	Mapping verso l’implementazione (coerente con L10)	5
3	Esercizio 2 — Backend NestJS	5
3.1	Nuova interfaccia <code>UserListItem</code> (FILE NUOVO)	5
3.2	Export dell’interfaccia (MODIFICA)	6
3.3	Mapping in <code>getUsers()</code> (MODIFICA)	6
3.4	Quadro completo delle rotte del controller utenti	6
4	Esercizio 3 — Frontend React	8
4.1	API client (FILE NUOVO)	8
4.2	Pagina elenco utenti (FILE NUOVO)	9
4.3	Pagina “Funzione non implementata” (FILE NUOVO)	11
4.4	Rotte (MODIFICA)	12
4.5	Voce di menu nella navbar (MODIFICA)	13
4.6	Approfondimento — come funziona il routing frontend	13
5	Configurazione Nx	14
6	Verifica finale	14
7	Note / limiti noti (fuori scope della consegna)	15
8	Elenco rapido dei file toccati	15

1 La consegna in breve

Introdurre la **gestione utenti limitata alla sola visualizzazione** degli utenti registrati:

- Pagina accessibile **solo agli ADMIN**.
- Tabella con colonne **id, name, email, role**.
- Pulsanti **Nuovo / Modifica / Elimina** (con conferma sulla cancellazione).
- Stile uguale alle pagine libri/autori/categorie già esistenti.
- Le pagine di **insert/edit NON sono richieste** (la cancellazione era opzionale, ma è stata comunque implementata e funzionante).

I tre esercizi.

1. **Esercizio 1** — modello ipertestuale **WebML** della pagina.
2. **Esercizio 2** — rotta **backend NestJS** che espone gli utenti.
3. **Esercizio 3** — pagina **frontend React**.

Architettura del progetto. Monorepo **Nx**, con `apps/api` (NestJS), `apps/ui` (React + Vite), e librerie `libs/server/{auth,users,books,security}`.

2 Esercizio 1 — Modello WebML (documentazione)

File creato: `Appello_20260629/docs/Esercizio1_WebML_GestioneUtenti.md`

Questo esercizio è solo documentale: non tocca il codice, ma **giustifica** le scelte di Es.2 ed Es.3 tramite il modello ipertestuale WebML. Di seguito il contenuto **integrale** del documento.

Progettazione model-driven dei contenuti ipertestuali (WebML). Riferimenti: slide L9 (notazione) e L10 (running example “Area libri” + mapping all’implementazione).

2.1 Premesse e requisiti

Dall’analisi dei requisiti (consegna):

- Funzionalità di **gestione utenti**, limitata alla **sola visualizzazione** degli utenti registrati.
- Accesso consentito **solo agli amministratori** → la pagina appartiene alla **site view privata dell’amministratore** (area *privata*: accesso tramite autenticazione + autorizzazione di ruolo ADMIN).
- La pagina mostra una **tabella** con `id`, `nome (name)`, `email`, `ruolo (role)`.
- Sono presenti i **pulsanti** per: inserimento nuovo utente, modifica utente, cancellazione (con conferma).
- Le pagine di inserimento/modifica **non** sono realizzate (i relativi link puntano a pagine non ancora sviluppate). La cancellazione è opzionale.

Entità coinvolta (dal modello dei dati): `User(id, name, email, passwordHash, role)`. Nella pubblicazione ipertestuale l’attributo `passwordHash` **non** viene mai esposto.

2.2 Progetto coarse — Area Utenti

Si individua una nuova **area** coesa, dedicata alla gestione degli utenti, all'interno della site view privata dell'amministratore (analoga alle aree “libri”, “autori”, “categorie”).

```

:.....:
: Area Utenti :
:-----:
: Core (User) :
: Delete (User) <-- opzionale :
: [Create (User)] <-- pagina non realizzata
: [Modify (User)] <-- pagina non realizzata
:.....:

```

- **Core(User)**: pubblicazione dei contenuti dell'entità core **User** → richiesta dell'esercizio.
- **Delete(User)**: operazione CUD di cancellazione (opzionale).
- **Create(User) / Modify(User)**: previste come funzioni dell'area, ma **non realizzate** in questo esercizio (i pulsanti sono comunque mostrati e i link puntano alle relative pagine).

Visibilità dell'area: *privata* (accesso previa autenticazione e con ruolo ADMIN).

2.3 Progetto dettagliato — suddivisione in pagine

L'area si suddivide nelle seguenti pagine (solo la prima è realizzata in questo esercizio):

```

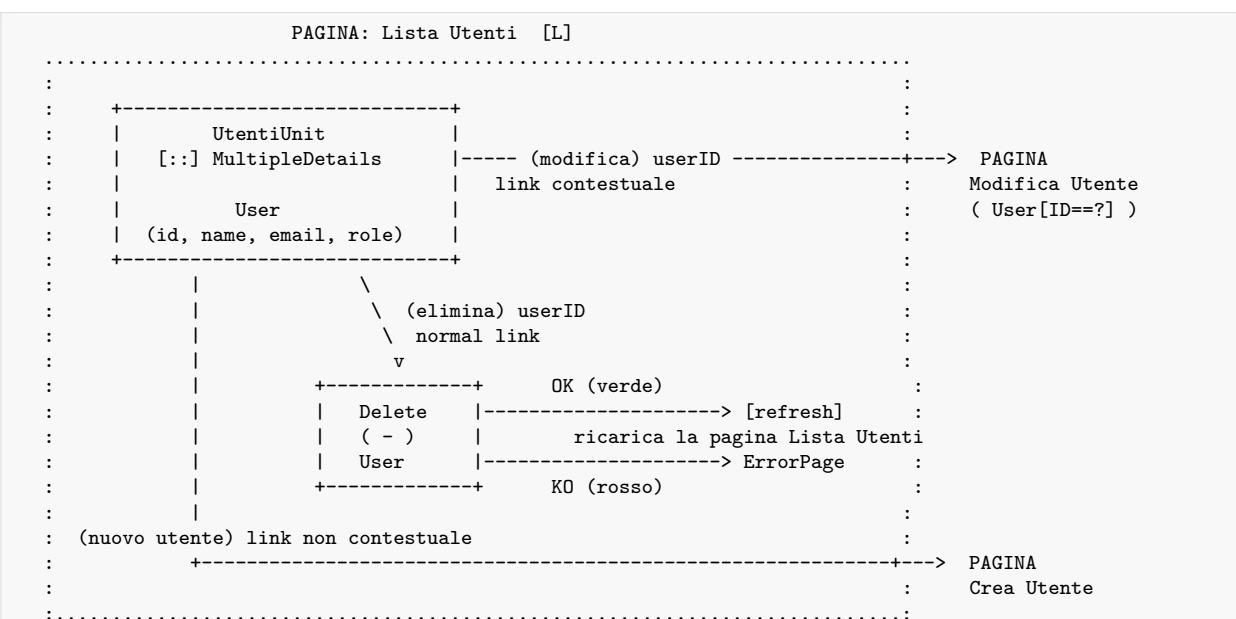
:.....: :.....: :.....:
: Lista Utenti [L] : : Crea Utente [L] : : Modifica Utente :
:-----: :-----: :-----:
: Core (User) : : Create (User) : : Modify (User) :
: Delete (User) : : (non realizzata) : : (non realizzata) :
:.....: :.....: :.....:

```

- **Lista Utenti** — pagina **Landmark** [L]: globalmente visibile, raggiungibile da ogni altra pagina della site view tramite la *navbar* (voce “Utenti”). È la pagina oggetto dell'esercizio.
- **Crea Utente / Modifica Utente**: pagine non realizzate (placeholder di destinazione dei link).

2.4 Specifica di dettaglio della pagina «Lista Utenti»

2.4.1 Diagramma WebML



LEGENDA
 [::] MultipleDetails unit (-) Delete operation unit
 ---> link ----- link contestuale (trasporta userID)
 OK = link verde (successo) KO = link rosso (fallimento)

2.4.2 View component — UtentiUnit (MultipleDetails)

Si usa una **MultipleDetails unit** perché la pagina pubblica **un insieme di istanze** dell'entità **User**, mostrando per ciascuna più attributi contemporaneamente (la tabella).

Proprietà	Valore
Tipo unità	MultipleDetails
Contenitore (sorgente)	entità User
Selettore	<i>nessuno</i> → tutte le istanze (l'admin vede tutti gli utenti)
Attributi pubblicati	id, name, email, role (mai passwordHash)
Parametri in input	nessuno
Parametri in output	l'insieme degli {id} degli utenti pubblicati; userID della riga selezionata per i link

2.4.3 Operazione — Delete(User) (opzionale)

Proprietà	Valore
Tipo	Delete operation unit
Entità	User
Normal link (in)	dalla UtentiUnit , trasporta lo userID dell'utente da eliminare
Link OK (verde)	torna a Lista Utenti (ricalcolo/refresh della MultipleDetails)
Link KO (rosso)	va a ErrorPage

La conferma (“con conferma”) è modellata come interazione che precede l'attivazione del *normal link* verso l'operazione **Delete**.

2.4.4 Link uscenti dalla pagina

Pulsante	Tipo di link	Destinazione	Contesto trasportato
Nuovo utente	link non contestuale	pagina Crea Utente	nessuno
Modifica	link contestuale	pagina Modifica Utente	userID (→ User [ID==?])
Elimina	normal link verso operazione Delete	Delete(User)	userID

2.5 Layout di pagina (navbar + utente corrente)

Come nel running example, il **Layout page** è un modulo [M] globale che definisce navbar e informazioni sull'utente autenticato. La voce di menu **“Utenti”** (verso la pagina *Lista Utenti* landmark) è visibile **solo** se l'utente corrente ha ruolo ADMIN.

```

:.....:
: Layout page           [M] :
:-----:
:   ( ) CurrentUser      :
:   |                    :
:   v                    :
:   [=] User [ID==?]     :   (Details dell'utente loggato, mostrato in navbar)
:.....:
: navbar -> { Catalogo | Autori | Categorie | **Utenti (solo ADMIN)** | Logout }

```

2.6 Mapping verso l'implementazione (coerente con L10)

Costrutto WebML	Implementazione
Pagina <i>Lista Utenti</i>	componente React <code>users.page.tsx</code>
View component <i>UtentiUnit</i> (visualizz.)	tabella renderizzata nel componente React
Caricamento dati dal backend	funzione <code>fetchUsers()</code> in <code>users.api.ts</code> → GET <code>/api/users</code> (solo ADMIN)
Operazione <i>Delete(User)</i>	funzione <code>deleteUser(id)</code> in <code>users.api.ts</code> → DELETE <code>/api/users/:id</code>
Link automatico (rendering)	<code>useEffect</code> che invoca <code>fetchUsers()</code> al mount
Pagina landmark <i>Utenti</i>	voce della navbar nel <code>app-layout.tsx</code> (visibile solo agli ADMIN)
Link contestuale <i>Modifica</i>	<code>navigate('/users/:id/edit')</code>
Link non contestuale <i>Nuovo utente</i>	<code>navigate('/users/new')</code>
ErrorPage (link KO)	gestione dello stato <code>error</code> nel componente

3 Esercizio 2 — Backend NestJS

Obiettivo: esporre la lista utenti **senza mai restituire** `passwordHash`, su una rotta protetta accessibile solo agli ADMIN (la rotta GET `/api/users` già esisteva).

3.1 Nuova interfaccia *UserListItem* (FILE NUOVO)

`libs/server/users/src/lib/interfaces/user-list-item.interface.ts`

```

1 import { UserRole } from '../dto/user-role.enum';
2
3 /**
4  * Proiezione "sicura" di un utente per la pubblicazione ipertestuale
5  * (View component UtentiUnit dell'Esercizio 1): espone solo gli attributi
6  * mostrati nella tabella, escludendo sempre 'passwordHash'.
7  */
8 export interface UserListItem {
9   id: number;
10  name: string;
11  email: string;

```

```

12   role: UserRole;
13 }

```

Perché: è la “proiezione sicura” dell’entità utente. Definisce un tipo che contiene **solo** i campi mostrati in tabella, garantendo a livello di tipo che `passwordHash` non venga propagato verso il frontend.

3.2 Export dell’interfaccia (MODIFICA)

`libs/server/users/src/index.ts` — aggiunta l’ultima riga:

```

1 export * from './lib/users.module';
2 export * from './lib/users.service';
3 export * from './lib/users.repository';
4 export * from './lib/user.entity';
5 export * from './lib/dto/user-role.enum';
6 export * from './lib/dto/create-user.dto';
7 export * from './lib/dto/update-user.dto';
8 export * from './lib/interfaces/user-list-item.interface'; // <-- AGGIUNTA

```

Perché: rende `UserListItem` importabile sia dal backend sia dal frontend (`import { UserListItem } from '@server/users'`), così client e server condividono lo stesso contratto di tipo.

3.3 Mapping in `getUsers()` (MODIFICA)

`libs/server/users/src/lib/users.service.ts`

```

1 import { UserListItem } from './interfaces/user-list-item.interface'; // <-- import aggiunto
2
3 async getUsers(role?: UserRole): Promise<UserListItem[]> { // <-- tipo di ritorno
4     cambiato
5     const users = await this.usersRepository.findAll(role);
6
7     if (role && users.length === 0) {
8         throw new NotFoundException('No users found with role ${role}');
9     }
10
11     // Mapping verso la proiezione "sicura": non esporre mai passwordHash.
12     return users.map(({ id, name, email, role }) => ({ id, name, email, role })); // <-- AGGIUNTA

```

Perché: prima il metodo restituiva le entità complete (incluso `passwordHash`). Ora il `.map()` estrae solo i quattro campi previsti → la rotta non espone più dati sensibili.

3.4 Quadro completo delle rotte del controller utenti

`libs/server/users/src/lib/users.controller.ts` — il controller è montato su `@Controller('users')` e il prefisso globale dell’app è `/api`, quindi tutte le rotte vivono sotto `/api/users`. Ecco **ogni rotta** con le relative protezioni:

Metodo + path	Guardie	Ruoli ammessi	Cosa fa	Usata?
GET <code>/users (opz. ?role=)</code>	JwtAuthGuard, RolesGuard	ADMIN	Lista utenti (proiezione sicura <code>UserListItem[]</code>)	Sì — alimenta la tabella

Metodo + path	Guardie	Ruoli ammessi	Cosa fa	Usata?
GET /users/me	JwtAuthGuard	qualsiasi loggato	Restituisce l'utente corrente dal token	indirettamente (navbar)
GET /users/interns	nessuna	pubblica	Placeholder (“API non implementata”)	no
GET /users/:id	JwtAuthGuard, RolesGuard	ADMIN, USER	Dettaglio utente	no
POST /users	nessuna	pubblica	Crea utente (accetta role)	no (preesistente)
PATCH /users/:id	JwtAuthGuard, RolesGuard	ADMIN	Aggiorna utente	no
DELETE /users/:id	JwtAuthGuard, RolesGuard	ADMIN	Cancella utente	Sì — pulsante “Elimina”

Importante: l'ordine di dichiarazione conta. GET /users/me e GET /users/interns sono dichiarati **prima** di GET /users/:id, altrimenti :id catturerebbe anche me/interns interpretandoli come id.

Le due rotte effettivamente sfruttate dalla pagina:

```

1 // 1) Lettura della lista (tabella)
2 @Get() // GET /api/users oppure /api/users?role=value
3 @UseGuards(JwtAuthGuard, RolesGuard)
4 @Roles(UserRole.ADMIN)
5 @ApiBearerAuth()
6 @ApiQuery({ name: 'role', required: false, enum: UserRole })
7 getUsers(@Query('role', new ParseEnumPipe(UserRole, { optional: true })) role?: UserRole) {
8   return this.serverUsersService.getUsers(role);
9 }
10
11 // 2) Cancellazione (pulsante Elimina)
12 @Delete('/:id') // DELETE /api/users/:id
13 @UseGuards(JwtAuthGuard, RolesGuard)
14 @Roles(UserRole.ADMIN)
15 @ApiBearerAuth()
16 removeUser(@Param('id', ParseIntPipe) id: number) {
17   return this.serverUsersService.removeUser(id);
18 }

```

Come funziona la protezione (catena di guardie):

1. JwtAuthGuard — legge l'header Authorization: Bearer <token>, valida il JWT e popola request.user. Senza token valido → **401 Unauthorized**.
2. RolesGuard — legge i ruoli richiesti impostati dal decoratore @Roles(...) (via metadata Reflector) e li confronta con il ruolo dell'utente nel token. Se non combaciano → **403 Forbidden**.
3. @Roles(UserRole.ADMIN) — non esegue logica: **annota** la rotta con i ruoli ammessi, che RolesGuard poi legge. È il pezzo che impone “solo ADMIN”.
4. @ApiBearerAuth() / @ApiQuery(...) — solo documentazione Swagger, nessun effetto a runtime.

Pipe sui parametri:

- ParseIntPipe su :id → converte e valida che l'id sia un intero (altrimenti 400).

- `ParseEnumPipe(UserRole, { optional: true })` sul query role → accetta solo valori validi dell'enum, ed è opzionale.

Flusso completo di una richiesta GET /api/users (end-to-end):

```
UsersPage (useEffect)
  -> fetchUsers() [users.api.ts]
  -> GET /api/users + Bearer token
      -> JwtAuthGuard (valida token, set request.user)
      -> RolesGuard (richiede ADMIN)
      -> ServerUsersController.getUsers()
      -> ServerUsersService.getUsers() [mappa a UserListItem, niente passwordHash]
      -> UsersRepository.findAll()
  <- JSON: UserListItem[]
  -> setUsers(...) -> render tabella
```

4 Esercizio 3 — Frontend React

Obiettivo: nuova pagina /users (solo ADMIN) con tabella e pulsanti, in stile identico alle altre pagine.

4.1 API client (FILE NUOVO)

apps/ui/src/features/users/users.api.ts

Codice completo del file (fedele all'originale):

```
1 import { UserListItem } from '@server/users';
2 import { handleApiError } from '../shared/utils.api';
3
4 const API_URL = 'http://localhost:3333/api';
5
6 function getAuthHeaders() {
7   const token = localStorage.getItem('access_token');
8
9   return {
10     'Content-Type': 'application/json',
11     Authorization: `Bearer ${token}`,
12   };
13 }
14
15 export async function fetchUsers(): Promise<UserListItem[]> {
16   const response = await fetch(`${API_URL}/users`, {
17     headers: getAuthHeaders(),
18   });
19
20   if (!response.ok) {
21     await handleApiError(response);
22   }
23
24   return response.json();
25 }
26
27 export async function deleteUser(id: number): Promise<void> {
28   const response = await fetch(`${API_URL}/users/${id}`, {
29     method: 'DELETE',
30     headers: getAuthHeaders(),
31   });
32
33   if (!response.ok) {
34     await handleApiError(response);
35   }
36 }
```

Perché: centralizza le chiamate HTTP (con token JWT negli header) e riusa il tipo `UserListItem` esportato dal backend. Coerente con gli altri `*.api.ts`.

4.2 Pagina elenco utenti (FILE NUOVO)

`apps/ui/src/features/users/users.page.tsx`

Punti chiave:

- Carica gli utenti con `fetchUsers()` dentro un `useEffect`, gestendo gli stati `loading` / `error`.
- Tabella con colonne **Id** / **Nome** / **Email** / **Ruolo** / **Azioni**.
- Pulsante “Nuovo utente” → naviga a `/users/new`.
- Pulsante “Modifica” per riga → naviga a `/users/:id/edit`.
- Pulsante “Elimina” per riga → `window.confirm` di conferma, poi `deleteUser(id)` e aggiornamento ottimistico della lista (`filter`).
- Stile riutilizzato da `../css/books.module.css`.

Codice completo del file:

```
1 import { useEffect, useState } from 'react';
2 import { useNavigate } from 'react-router-dom';
3 import clsx from 'clsx';
4 import book_styles from '../css/books.module.css';
5 import { UserListItem } from '@server/users';
6 import { deleteUser, fetchUsers } from './users.api';
7
8 export function UsersPage() {
9   const navigate = useNavigate();
10  const [users, setUsers] = useState<UserListItem[]>([]);
11  const [loading, setLoading] = useState(true);
12  const [error, setError] = useState<string | null>(null);
13
14  useEffect(() => {
15    fetchUsers()
16      .then(setUsers)
17      .catch((err) => setError(err.message))
18      .finally(() => setLoading(false));
19  }, []);
20
21  async function handleDelete(id: number) {
22    const confirmed = window.confirm('Vuoi davvero cancellare questo utente?');
23
24    if (!confirmed) return;
25
26    try {
27      await deleteUser(id);
28      setUsers((current) => current.filter((user) => user.id !== id));
29    } catch (err: any) {
30      setError(err.message);
31    }
32  }
33
34  if (loading) {
35    return (
36      <main className={book_styles.page}>
37        <section className={book_styles.card}>
38          <p className={book_styles.message}>Caricamento utenti...</p>
39        </section>
40      </main>
41    );
42  }
```

```

42 }
43
44 if (error) {
45   return (
46     <main className={book_styles.page}>
47       <section className={book_styles.card}>
48         <p className={book_styles.error}>{error}</p>
49       </section>
50     </main>
51   );
52 }
53
54 return (
55   <main className={book_styles.page}>
56     <section className={clsx(book_styles.card, book_styles.cardLarge)}>
57       <header className={book_styles.header}>
58         <h1 className={book_styles.title}>[user] Utenti</h1>
59         <p className={book_styles.subtitle}>
60           Elenco degli utenti registrati nella libreria.
61         </p>
62       </header>
63
64       <button
65         className={book_styles.button}
66         onClick={() => navigate('/users/new')}
67       >
68         Nuovo utente
69       </button>
70
71       {users.length === 0 ? (
72         <p className={book_styles.message}>Nessun utente disponibile.</p>
73       ) : (
74         <div className={book_styles.tableWrapper}>
75           <table className={book_styles.table}>
76             <thead>
77               <tr>
78                 <th className={book_styles.th}>Id</th>
79                 <th className={book_styles.th}>Nome</th>
80                 <th className={book_styles.th}>Email</th>
81                 <th className={book_styles.th}>Ruolo</th>
82                 <th className={book_styles.th}>Azioni</th>
83               </tr>
84             </thead>
85
86             <tbody>
87               {users.map((user) => (
88                 <tr key={user.id} className={book_styles.row}>
89                   <td className={book_styles.td}>{user.id}</td>
90                   <td className={book_styles.titleCell}>{user.name}</td>
91                   <td className={book_styles.td}>{user.email}</td>
92                   <td className={book_styles.td}>{user.role}</td>
93                   <td className={book_styles.td}>
94                     <button
95                       className={book_styles.secondaryButton}
96                       onClick={() => navigate('/users/${user.id}/edit')}
97                     >
98                       Modifica
99                     </button>
100
101                     <button
102                       className={book_styles.dangerButton}
103                       onClick={() => handleDelete(user.id)}
104                     >
105                       Elimina

```

```

106         </button>
107     </td>
108 </tr>
109 }}
110 </tbody>
111 </table>
112 </div>
113 }}
114 </section>
115 </main>
116 );
117 }

```

Spiegazione riga per riga delle scelte:

- `const [users, setUsers] = useState<UserListItem[]>([])` — lo stato è **tipizzato** con il contratto condiviso `UserListItem`: il compilatore garantisce che si accede solo a `id/name/email/role`.
- `useEffect(..., [])` con array di dipendenze vuoto → il fetch parte **una sola volta** al montaggio del componente.
- `.then(setSomething).catch(...).finally(...)` → pattern a tre stati (dati / errore / caricamento), identico alle altre pagine del progetto.
- `handleDelete` → **conferma** con `window.confirm` (requisito della consegna), poi cancella sul server e infine aggiorna lo stato locale con `filter((user) => user.id !== id)` (**aggiornamento ottimistico**: non si rifà la fetch, si rimuove la riga dallo stato).
- Rendering condizionale: `loading` → messaggio; `error` → messaggio d'errore; lista vuota → “Nessun utente disponibile”; altrimenti → tabella.
- Le classi CSS (`page`, `card`, `cardLarge`, `table`, `th`, `td`, `titleCell`, `secondaryButton`, `dangerButton`, ...) sono **riusate** da `books.module.css`, così lo stile è identico alle pagine libri/autori/categorie.

Nota sull'evoluzione: nella prima versione i pulsanti “Nuovo” e “Modifica” erano **disabled** con `title="Funzione non disponibile"`. In un secondo momento sono stati resi cliccabili e fatti puntare alla paginetta “Funzione non implementata” (vedi sezione successiva).

4.3 Pagina “Funzione non implementata” (FILE NUOVO)

`apps/ui/src/features/users/user-not-implemented.page.tsx`

```

1 import { useNavigate } from 'react-router-dom';
2 import clsx from 'clsx';
3 import book_styles from '../css/books.module.css';
4
5 export function UserNotImplementedPage() {
6     const navigate = useNavigate();
7     return (
8         <main className={book_styles.page}>
9             <section className={clsx(book_styles.card, book_styles.cardSmall)}>
10                 <h1 className={book_styles.title}>[WIP] Funzione non implementata</h1>
11                 <p className={book_styles.message}>
12                     La creazione e la modifica degli utenti non sono ancora disponibili.
13                     La gestione utenti è attualmente limitata alla sola visualizzazione.
14                 </p>
15                 <button className={book_styles.button} onClick={() => navigate('/users')}>
16                     Torna agli utenti
17                 </button>
18             </section>
19         </main>

```

```
20 );
21 }
```

Perché: la consegna non richiede insert/edit. Invece di lasciare pulsanti morti, i pulsanti Nuovo/Modifica portano a questa pagina che dichiara esplicitamente che la funzione non è implementata, mantenendo la UX coerente.

4.4 Rotte (MODIFICA)

apps/ui/src/app/app.tsx — sono stati aggiunti **2 import** e **3 rotte** (le righe marcate // <- AGGIUNTA). Di seguito il file completo per vedere le rotte nel loro contesto:

```
1 import { BooksPage } from '../features/books/books.page';
2 import { Routes, Route, Navigate } from 'react-router-dom';
3 import { LoginPage } from '../features/auth/login.page';
4 import { ProtectedRoute } from '../features/auth/protected-route';
5 import { LogoutPage } from '../features/auth/logout.page';
6 import { CreateBookPage } from '../features/books/create-book.page';
7 import { EditBookPage } from '../features/books/edit-book.page';
8 import { AppLayout } from '../features/layouts/app-layout';
9 import { HomeModulePage } from '../features/books/home-module.page';
10 import { HomeTailwindPage } from '../features/books/home-tailwind.page';
11 import { HomeBootstrapPage } from '../features/books/home-bootstrap.page';
12 import { AuthorsPage } from '../features/authors/authors.page';
13 import { CreateAuthorPage } from '../features/authors/create-author.page';
14 import { EditAuthorPage } from '../features/authors/edit-author.page';
15 import { CategoriesPage } from '../features/categories/categories.page';
16 import { CreateCategoryPage } from '../features/categories/create-category.page';
17 import { EditCategoryPage } from '../features/categories/edit-category.page';
18 import { UsersPage } from '../features/users/users.page'; // <-
    AGGIUNTA
19 import { UserNotImplementedPage } from '../features/users/user-not-implemented.page'; // <-
    AGGIUNTA
20
21 export function App() {
22   return (
23     <Routes>
24       <Route path="/" element={<Navigate to="/books" replace />} />
25       <Route path="/login" element={<LoginPage />} />
26       <Route path="/logout" element={<LogoutPage />} />
27
28       <Route
29         element={
30           <ProtectedRoute>
31             <AppLayout />
32           </ProtectedRoute>
33         }
34       >
35         <Route path="/books" element={<BooksPage />} />
36         <Route path="/books/new" element={<CreateBookPage />} />
37         <Route path="/books/:id/edit" element={<EditBookPage />} />
38         <Route path="/authors" element={<AuthorsPage />} />
39         <Route path="/authors/new" element={<CreateAuthorPage />} />
40         <Route path="/authors/:id/edit" element={<EditAuthorPage />} />
41         <Route path="/categories" element={<CategoriesPage />} />
42         <Route path="/categories/new" element={<CreateCategoryPage />} />
43         <Route path="/categories/:id/edit" element={<EditCategoryPage />} />
44         <Route path="/users" element={<UsersPage />} /> /* <- AGGIUNTA */
45       </Route>
46       <Route path="/users/new" element={<UserNotImplementedPage />} /> /* <- AGGIUNTA */
47       <Route path="/users/:id/edit" element={<UserNotImplementedPage />} /> /* <- AGGIUNTA */
48     </Routes>
49   );
50 }
```

```

47     </Route>
48
49     <Route path="/home-module" element={<HomeModulePage />} />
50     <Route path="/home-tailwind" element={<HomeTailwindPage />} />
51     <Route path="/home-bootstrap" element={<HomeBootstrapPage />} />
52   </Routes>
53 );
54 }
55
56 export default App;

```

Perché: le tre rotte utenti sono inserite **dentro** il blocco protetto da `<ProtectedRoute>` `<AppLayout/></ProtectedRoute>` (richiedono login e mostrano la navbar). `/users/new` e `/users/:id/edit` riusano la stessa pagina “non implementata”. Le rotte pubbliche (`/login`, `/logout`, le home demo) restano fuori dal blocco protetto.

4.5 Voce di menu nella navbar (MODIFICA)

`apps/ui/src/features/layouts/app-layout.tsx` — aggiunta la voce “Utenti” nella sezione `navLinks` (riga marcata). Contesto del blocco modificato:

```

1 <div className={styles.navLinks}>
2   <button onClick={() => navigate('/books')}>Catalogo</button>
3   <button onClick={() => navigate('/books/new')}>Nuovo libro</button>
4   <button onClick={() => navigate('/authors')}>Autori</button>
5   <button onClick={() => navigate('/categories')}>Categorie</button>
6   {user?.role === 'ADMIN' && (                                     {/* <-- AGGIUNTA */}
7     <button onClick={() => navigate('/users')}>Utenti</button>      {/* <-- AGGIUNTA */}
8   )}                                                                {/* <-- AGGIUNTA */}
9
10  <div className={styles.userSection}>
11    {/* ... pulsante utente + dropdown logout (invariati) ... */}
12  </div>
13 </div>

```

Il ruolo arriva dallo stato `user`, popolato al mount da `fetchCurrentUser()` (GET `/api/users/me`):

```

1 const [user, setUser] = useState(null);
2
3 useEffect(() => {
4   fetchCurrentUser()
5     .then(setUser)
6     .catch((err) => setUser(null));
7 }, []);

```

Perché: la voce “Utenti” compare nel menu solo se l’utente loggato è **ADMIN** (`user?.role === 'ADMIN'`), in linea con la consegna (pagina riservata agli ADMIN).

4.6 Approfondimento — come funziona il routing frontend

Le tre rotte utenti sono **annidate** dentro una rotta “wrapper” senza `path`:

```

1 <Route
2   element={
3     <ProtectedRoute>
4       <AppLayout />
5     </ProtectedRoute>
6   }
7 >
8   {/* ... books, authors, categories ... */}
9   <Route path="/users" element={<UsersPage />} />

```

```

10 <Route path="/users/new" element={<UserNotImplementedPage />} />
11 <Route path="/users/:id/edit" element={<UserNotImplementedPage />} />
12 </Route>

```

Cosa implica, passo per passo:

1. **Rotta wrapper (layout route):** non ha path, serve solo a raggruppare le rotte figlie e ad applicare loro lo stesso “guscio”.
2. `<ProtectedRoute>` — verifica l’autenticazione (presenza/validità del token); se l’utente non è loggato reindirizza al login. Tutte le rotte figlie ereditano questa protezione → `/users`, `/users/new`, `/users/:id/edit` sono **tutte** accessibili solo da utenti autenticati.
3. `<AppLayout>` — disegna la navbar e poi un `<Outlet/>`: è dentro l’`Outlet` che React Router inietta la pagina figlia corrente. Per questo la navbar resta fissa mentre cambia solo il contenuto.
4. `navigate(...)` — la navigazione è programmatica (hook `useNavigate`):
 - `navigate('/users')` dalla navbar (solo ADMIN);
 - `navigate('/users/new')` dal pulsante “Nuovo utente”;
 - `navigate('/users/${user.id}/edit')` dal pulsante “Modifica” (URL dinamico con id);
 - `navigate('/users')` dal pulsante “Torna agli utenti” nella pagina non implementata.
5. `:id` è un parametro dinamico: qualsiasi valore (`/users/7/edit`, `/users/42/edit`, ...) matcha la stessa rotta. Qui punta alla pagina “non implementata”, quindi l’id non viene letto; in una vera edit si leggerebbe con `useParams()`.

Livelli di protezione per `/users` (tre livelli):

Livello	Meccanismo	Cosa blocca
Navbar	<code>user?.role === 'ADMIN'</code>	nasconde il link ai non-ADMIN
Frontend route	<code><ProtectedRoute></code>	blocca gli utenti non autenticati
Backend	<code>JwtAuthGuard + RolesGuard + @Roles(ADMIN)</code>	401/403 su accesso non autorizzato (anche via URL diretto)

Nota: la rotta frontend `/users` controlla solo l’autenticazione, non il ruolo; la vera barriera sul ruolo è il **backend**, che risponde 403 a un non-ADMIN che forzasse l’URL.

5 Configurazione Nx

Avendo introdotto una dipendenza nuova `ui` → `@server/users` (per importare il tipo `UserListItem`), è stato eseguito `nx sync` per aggiornare i *project references* di TypeScript.

6 Verifica finale

- `nx run-many -t build -p api ui` → **build OK** per entrambi i progetti.
- `nx lint`:
 - `ui` → solo *warning* preesistenti (`any` + emoji), identici a `categories.page`.

- @server/users:lint → fallisce per una **dipendenza circolare PREESISTENTE** @server/users ↔ @server/security (controller ↔ roles.guard). **Non introdotta da noi.**

7 Note / limiti noti (fuori scope della consegna)

- La pagina /users è protetta da ProtectedRoute (autenticazione) ma **non** ricontrolla il ruolo a livello di rotta frontend: un non-ADMIN che digita /users a mano riceve comunque **403 dal backend** (la pagina mostra l'errore). La voce in navbar è già nascosta ai non-ADMIN.
- POST /users e register restano **pubbliche** e accettano role (possibile *privilege escalation*) — comportamento preesistente, fuori dallo scope.
- Dipendenza circolare tra librerie server — preesistente.

8 Elenco rapido dei file toccati

File	Tipo	Esercizio
docs/Esercizio1_WebML_GestioneUtenti.md	nuovo	Es.1
libs/server/users/src/lib/interfaces/user-list-item.interface.ts	nuovo	Es.2
libs/server/users/src/index.ts	modificato	Es.2
libs/server/users/src/lib/users.service.ts	modificato	Es.2
apps/ui/src/features/users/users.api.ts	nuovo	Es.3
apps/ui/src/features/users/users.page.tsx	nuovo	Es.3
apps/ui/src/features/users/user-not-implemented.page.tsx	nuovo	Es.3
apps/ui/src/app/app.tsx	modificato	Es.3
apps/ui/src/features/layouts/app-layout.tsx	modificato	Es.3